Reviewer Pack — Hook-Length Ratio Bounds Monotone Circuit Size for Permanent

Entry c99c02cb9842 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 21:10:02 UTC

Hook-Length Ratio Bounds Monotone Circuit Size for Permanent

Entry ID: `c99c02cb9842` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-12 21:10:02 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Hook-Length Formula) **Field B** (complexity object): Monotone Circuit Size for Permanent

Statement:

> For every n ≥ 1, the ratio of standard Young tableaux counts for the permanent's Young diagram λ_n to the determinant's μ_n satisfies |SYT(λ_n)| / |SYT(μ_n)| ≥ 2^{n/2}. This ratio equals the dimension of the symmetric power of the defining representation divided by the exterior power dimension.

Rationale (proposer's reasoning):

> The hook-length formula quantifies representation dimensions tied to permutation symmetries. Permanent's symmetric structure forces exponentially larger tableau counts than determinant's alternating structure, suggesting inherent complexity in monotone computation.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e319ec3d9b3dd0b8`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import product

def memoize(f):
    cache = {}
    def memoized_func(*args):
        if args in cache:
            return cache[args]
        result = f(*args)
        cache[args] = result
        return result
    return memoized_func

@memoize
def hook_length_formula(n, k):
    if n == 0 or k == 0:
        return 1
    return (n - k + 1) * hook_length_formula(n - 1, k - 1) / (n + 1)

def hook_length_tableau_count(n):
    total = 1
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            total *= hook_length_formula(i, j)
    return total

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        λ_n = (n, n)
        μ_n = tuple(range(1, n + 1))
```

```
        SYT_λ_n = hook_length_tableau_count(n)
        SYT_μ_n = hook_length_tableau_count(n)

        ratio = SYT_λ_n / SYT_μ_n
        total_metric_value += ratio
        instances_tested += 1

        if ratio < math.pow(2, n / 2):
            conjecture_holds = False
            counterexample = f"n={n}, λ_n={λ_n}, μ_n={μ_n}, SYT_λ_n={SYT_λ_n}, SYT_μ_n={SYT_μ_n},

    return {
        "metric_name": "Hook-Length Ratio",
        "metric_value": total_metric_value / len(n_values),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        from sympy import primerange
        seeds = list(primerange(2, 50))[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}**}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5fc5efa6.py", line 79, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5fc5efa6.py", line 55, in run_trial
    ratio = SYT_λ_n / SYT_μ_n
            ~~~~~~~~^~~~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed with division by zero, preventing evaluation of the ratio. The error suggests a potential flaw in SYT_μ_n calculation or invalid input for some n. | next: Debug the SYT_μ_n computation to verify if μ_n's Young diagram is valid for all n, and check if division by zero occurs for specific cases.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	67892
2	preregistration	ollama_remote	qwen3:8b	0	0	24023
3	novelty	ollama_remote	qwen3:8b	0	0	22427
4	novelty	ollama_remote	qwen3:8b	0	0	15413
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11794
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9231
7	judge	ollama_remote	qwen3:8b	0	0	22908

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 173688 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/c99c02cb9842.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c99c02cb9842.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c99c02cb9842.tar.gz` (if generated)